

Lab1: JOS启动和初始化

Lab 1

- ▶ Bootloader工作流程
- ▶ Kernel加载

JOS的启动流程

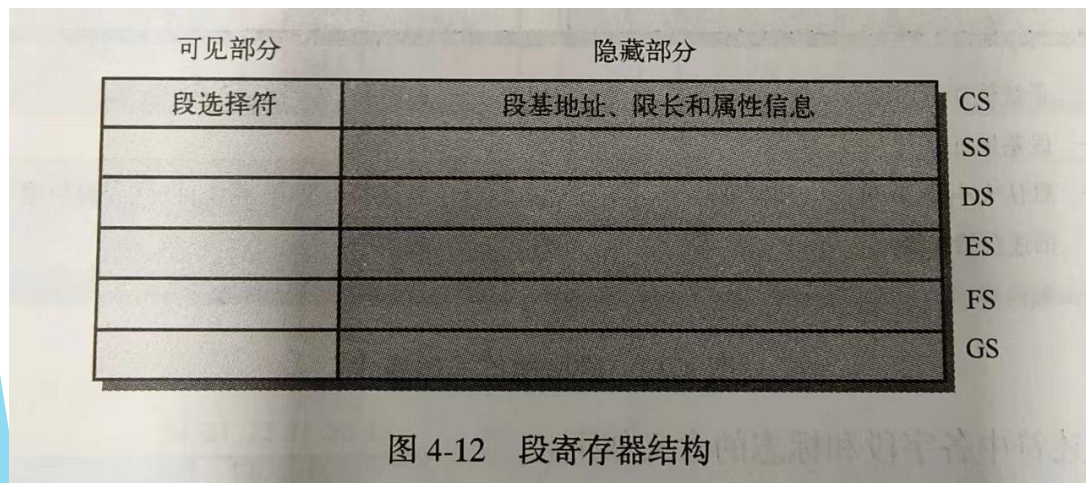
- ▶ 机器加电，BIOS自检
- ▶ 从磁盘的第一个扇区加载boot loader（检查扇区尾）
- ▶ Bootloader进行的工作
 - ▶ 初始化DS、ES、SS段寄存器
 - ▶ 开启A20地址线
 - ▶ 切换CPU至保护模式并启动GDT
 - ▶ 使用GDT表重新初始化段寄存器
 - ▶ 从磁盘的第二个扇区加载内核
 - ▶ 首先加载ELF文件头，获取文件的长度和各个程序块的信息
 - ▶ 加载kernel的各个模块（.text, .data等）
- ▶ 跳转至内核入口

在整个启动阶段中，CPU模式是如何切换的？

加载GDT与开启保护模式是否可以调换？

`ljmp $PROT_MODE_CSEG, $protcseg`的作用

- ▶ Boot.S 中的第55行，为什么要跳到下一行？
- ▶ `Ljmp`操作数除了要跳转到的地址外，还指定要切换到地址段
- ▶ `Ljmp`是更改CS寄存器的方法之一，CS寄存器需要包含GDT中代码段的选
择子
- ▶ 清空流水线



```
# Jump to next instruction, but in 32-bit code segment.
# Switches processor into 32-bit mode.
ljmp    $PROT_MODE_CSEG, $protcseg

.code32                                # Assemble for 32-bit mode
protcseg:
# Set up the protected-mode data segment registers
movw    $PROT_MODE_DSEG, %ax          # Our data segment selector
movw    %ax, %ds                       # -> DS: Data Segment
movw    %ax, %es                       # -> ES: Extra Segment
movw    %ax, %fs                       # -> FS
movw    %ax, %gs                       # -> GS
movw    %ax, %ss                       # -> SS: Stack Segment
```

不同地址的辨析

▶ 物理地址 & 虚拟地址

- ▶ 开启保护模式之前使用的是物理地址，之后开始使用虚拟地址，此时虚拟地址的寻址模式是段模式
- ▶ 在kernel启动初期，会把高的虚拟地址映射到低物理地址

▶ 链接地址 & 加载地址

- ▶ 加载地址是加载到内存中的地址，可能是虚拟地址也可能为物理地址，加载地址的信息是保存在header里面
- ▶ 链接地址是编译器使用的，链接不同的目标文件
- ▶ 在编译过程中指定，是静态的
- ▶ 在实验中，bootloader阶段链接地址和加载地址一样，但是在内核加载后就不一样了

Kernel的一些地址

- ▶ Kernel的链接地址和加载地址分别是多少?
 - ▶ 哪里定义了Kernel的链接地址和加载地址?
 - ▶ https://blog.csdn.net/weixin_43083491/article/details/127095711
- ▶ 能不能修改链接地址?
- ▶ Kernel的入口地址是多少?

mov \$relocated, %eax; jmp *%eax的作用?

- ▶ 在entry.S文件的67-69行

```
64      # Now paging is enabled, but we're still running at a low EIP
65      # (why is this okay?).  Jump up above KERNBASE before entering
66      # C code.
67      mov $relocated, %eax
68      jmp *%eax
69      relocated:
70
71      # Clear the frame pointer register (EBP)
72      # so that once we get into debugging C code,
73      # stack backtraces will be terminated properly.
74      movl    $0x0,%ebp          # nuke frame pointer
```

- ▶ 开启页表后要跳转到高地址空间。
- ▶ Kernel的加载地址是低地址，入口也是低地址，但是后续的运行需要在高地址空间中，所以需要跳转

VGA buffer的原理

- ▶ 在 VGA 兼容文字模式下，显示器的显示缓冲区被映射到一段特定区域的内存上。通过对这段内存的写入就可以在显示器上显示字符。
- ▶ Linux 会维护一片比较大的缓冲区，这样可以回滚找到之前的记录
- ▶ 在这段内存中，每个字符用一个 16 - bit 字节来代表。低字节的 8 bit 代表字符编码，高字节的 8 bit 代表字符颜色、背景颜色和 blink

Attribute								Character							
7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
Blink	Background color			Foreground color				Code point							

JOS如何处理键盘输入信号

- ▶ JOS怎么感知到用户“按下了键盘”并且知道“按下了哪个键”
- ▶ 中断？轮询？

键盘信号到字符是如何映射的?

- ▶ 键盘输入信号是如何变成屏幕上的字符的?
- ▶ 如何改键? 比如a和b互换?

```
246 static uint8_t normalmap[256] =
247 {
248     NO,    0x1B, '1', '2', '3', '4', '5', '6', // 0x00
249     '7', '8', '9', '0', '-', '=', '\b', '\t',
250     'q', 'w', 'e', 'r', 't', 'y', 'u', 'i', // 0x10
251     'o', 'p', '[', ']', '\n', NO, 'a', 's',
252     'd', 'f', 'g', 'h', 'j', 'k', 'l', ';', // 0x20
253     '\'', '`', NO, '\\', 'z', 'x', 'c', 'v',
254     'b', 'n', 'm', ',', '.', '/', NO, '*', // 0x30
255     NO, ' ', NO, NO, NO, NO, NO, NO,
256     NO, NO, NO, NO, NO, NO, NO, '7', // 0x40
257     '8', '9', '-', '4', '5', '6', '+', '1',
258     '2', '3', '0', '.', NO, NO, NO, NO, // 0x50
259     [0xC7] = KEY_HOME,      [0x9C] = '\n' /*KP_Enter*/,
260     [0xB5] = '/' /*KP_Div*/, [0xC8] = KEY_UP,
261     [0xC9] = KEY_PGUP,      [0xCB] = KEY_LF,
262     [0xCD] = KEY_RT,        [0xCF] = KEY_END,
263     [0xD0] = KEY_DN,        [0xD1] = KEY_PGDN,
264     [0xD2] = KEY_INS,       [0xD3] = KEY_DEL
265 };
266
267 static uint8_t shiftmap[256] =
268 {
269     NO,    033, '!', '@', '#', '$', '%', '^', // 0x00
270     '&', '*', '(', ')', '-', '+', '\b', '\t',
271     'Q', 'W', 'E', 'R', 'T', 'Y', 'U', 'I', // 0x10
272     'O', 'P', '{', '}', '\n', NO, 'A', 'S',
273     'D', 'F', 'G', 'H', 'J', 'K', 'L', ';', // 0x20
274     '"', '~', NO, '|', 'Z', 'X', 'C', 'V',
275     'B', 'N', 'M', '<', '>', '?', NO, '*', // 0x30
276     NO, ' ', NO, NO, NO, NO, NO, NO,
277     NO, NO, NO, NO, NO, NO, NO, '7', // 0x40
278     '8', '9', '-', '4', '5', '6', '+', '1',
279     '2', '3', '0', '.', NO, NO, NO, NO, // 0x50
280     [0xC7] = KEY_HOME,      [0x9C] = '\n' /*KP_Enter*/,
281     [0xB5] = '/' /*KP_Div*/, [0xC8] = KEY_UP,
282     [0xC9] = KEY_PGUP,      [0xCB] = KEY_LF,
283     [0xCD] = KEY_RT,        [0xCF] = KEY_END,
284     [0xD0] = KEY_DN,        [0xD1] = KEY_PGDN,
285     [0xD2] = KEY_INS,       [0xD3] = KEY_DEL
286 };
```

在cons_init之前，能否调用cprintf? cons_init做了什么?

- ▶ Init的时候初始化console, console会初始化一系列硬件，如果在这之前调用cprintf会没有输出，而且会报错
- ▶ cga_init() 初始化了VGA buffer指针

```
SEAX=00000000 EBX=00000084 ECX=00000753 EDX=00000001
ESI=00000000 EDI=f0111308 EBP=f010fed8 ESP=f010feb0
EIP=f010051e EFL=00000012 [----A--] CPL=0 II=0 A20=1 SMM=0 HLT=0
ES =0010 00000000 ffffffff 00cf9300 DPL=0 DS [-WA]
CS =0008 00000000 ffffffff 00cf9a00 DPL=0 CS32 [-R-]
SS =0010 00000000 ffffffff 00cf9300 DPL=0 DS [-WA]
DS =0010 00000000 ffffffff 00cf9300 DPL=0 DS [-WA]
FS =0010 00000000 ffffffff 00cf9300 DPL=0 DS [-WA]
GS =0010 00000000 ffffffff 00cf9300 DPL=0 DS [-WA]
LDT=0000 00000000 0000ffff 00008200 DPL=0 LDT
TR =0000 00000000 0000ffff 00008b00 DPL=0 TSS32-busy
GDT= 00007c4c 00000017
IDT= 00000000 000003ff
CR0=80010011 CR2=00000000 CR3=00112000 CR4=00000000
DR0=00000000 DR1=00000000 DR2=00000000 DR3=00000000
DR6=ffff0ff0 DR7=00000400
EFER=0000000000000000
Triple fault. Halting for inspection via QEMU monitor.
```

Thanks